

LeRobot数据集格式

王笑然

今天创建

- LeRobot 数据集是一个高度结构化的格式，支持从 [Hugging Face Hub](#) 或本地文件夹加载数据。

主要组成:

- `hf_dataset` : Hugging Face dataset (基于 Arrow/parquet 格式)。
 - 内容:
 - `observation` : 包含机器人的观测数据，例如:
 - `observation.images.cam_high` : 高分辨率摄像头的视频帧，格式为 `VideoFrame`，包含视频路径和时间戳。
 - `observation.state` : 机器人的关节位置等状态信息。
 - `action` : 机器人的动作数据，例如关节的目标位置。
 - `episode_index` : 当前样本所属的 episode 索引。
 - `frame_index` : 当前样本在其 episode 中的帧索引。
 - `timestamp` : 样本在 episode 中的时间戳。
 - `next.done` : 布尔值，表示是否为 episode 的最后一帧。
 - `index` : 在整个数据集全局索引。

episode: 一个完整的任务序列或交互过程，例如从机器人开始移动手臂到成功抓取物体并返回的整个过程可以被视为一个 episode。

`episode_index` & `frame_index` 示例:

假设我们有一个数据集，包含 3 个 episode，每个 episode 包含 5 个时间步。数据集的结构可能如下:

表格				复制
样本编号	episode_index	frame_index	观测 (observation)	动作 (action)
0	0	0	观测 1	动作 1
1	0	1	观测 2	动作 2
2	0	2	观测 3	动作 3
3	0	3	观测 4	动作 4
4	0	4	观测 5	动作 5
5	1	0	观测 6	动作 6
6	1	1	观测 7	动作 7
7	1	2	观测 8	动作 8
8	1	3	观测 9	动作 9
9	1	4	观测 10	动作 10
10	2	0	观测 11	动作 11
11	2	1	观测 12	动作 12
12	2	2	观测 13	动作 13
13	2	3	观测 14	动作 14
14	2	4	观测 15	动作 15

- `episode_data_index`
 - 内容:
 - `from` : 每个 episode 的起始帧索引。
 - `to` : 每个 episode 的结束帧索引。
 - `stats`
 - 内容: 数据集的统计信息，例如每个特征的最大值、均值、最小值和标准差。
 - `info`
 - 内容: 数据集的元数据，例如:
 - `codebase_version` : 创建数据集时使用的代码库版本。
 - `fps` : 数据集的帧率。
 - `video` : 是否使用视频格式存储帧。
 - `encoding` : 视频编码的参数。
 - `videos_dir`
 - 内容: 存储视频文件或图像文件的路径。
 - `camera_keys`
 - 内容: 用于访问摄像头数据的键值列表，例如 `["observation.images.cam_high", ...]`。

- 存放在 Huggingface 上的 LeRobot 训练数据

假如: ACT 模型有一个训练数据是 [aloha_sim_insertion_human](#)，除了数据内容和格式的说明文件，最重要的文件是 [train-00000-of-00001.parquet](#)

我们把这个 parquet 数据文件下载，然后用 [parquet-cli](#) 探视这个 parquest 文件的内容。

```
1 $ parq train-00000-of-00001.parquetMetadata
2 <pyarrow._parquet.FileMetaData object at 0x120dcf470>
3 created_by: parquet-cpp-arrow version 16.0.0
4 num_columns: 9
5 num_rows: 17000
6 num_row_groups: 17
7 format_version: 2.6
8 serialized_size: 19910
```

如上所示，文件中有 17 groups 数据，每个 group 包含 1000 行数据，总共 17000 行数据，每行数据包含 9 列。

```
1 $ parq train-00000-of-00001.parquet --schemaSchema
2 <pyarrow._parquet.ParquetSchema object at 0x136082740>
3 required group field_id=-1 schema {
4   optional group field_id=-1 observation.images.top {
5     optional binary field_id=-1 path (String);
6     optional float field_id=-1 timestamp;
7   } optional group field_id=-1 observation.state (List){
8     repeated group field_id=-1 list {
9       optional float field_id=-1 element;
10    } optional group field_id=-1 action (List){
11      repeated group field_id=-1 list {
12        optional float field_id=-1 element;
13      } optional int64 field_id=-1 episode_index;
14      optional int64 field_id=-1 frame_index;
15      optional float field_id=-1 timestamp;
16      optional boolean field_id=-1 next.done;
17      optional int64 field_id=-1 index;
18    }
```

如上所示，每行数据包含的 9 列，分别是，

- observation.images.top["path"],
- observation.images.top["timestamp"],
- observation.state,
- action,
- episode_index,
- frame_index,
- timestamp,
- next.done,
- index.

```
1 $ parq train-00000-of-00001.parquet --head 2
2 observation.images.top \
3 {'path': 'videos/observation.images.top_episod...
4 1 {'path': 'videos/observation.images.top_episod...
5 observation.state \
6 0 [0.0, -0.96, 1.16, 0.0, -0.3, 0.0, 0.0, 0.0, ...
7 1 [-0.0010152064, -0.9372796, 1.160248, -0.00504...
8 action episode_index \
9 0 [-0.012271847, -0.9096506, 1.1596895, -0.00613... 0
10 1 [-0.012271847, -0.90658265, 1.1627575, -0.0061... 0
11 frame_index timestamp next.done index
12 0 0 0.00 False 0
13 1 1 0.02 False 1
```

如上所示，提取出 train-00000-of-00001.parquet 文件中的前两行看看，确实与 schema 吻合。

之所以选择 parquet 作为存放文件的格式，大概率是因为 Huggingface 存放训练数据的文件系统，使用的是 Hadoop 的 HDFS。而 parquet 不仅能够支持 Hadoop 的数据处理流程，而且它可以读取文件的局部数据，而并不需要在内存中打开数据全文，所以读取数据的效率很高。



1 人点赞



输入评论

